SQL Fundamentals

Database Management and Structured Querying

SQL Fundamentals serves as an essential introduction to the ubiquity of databases in the digital landscape, particularly within cyber security. This module transitions from the theoretical understanding of **Relational** and **Non-Relational** databases to the practical application of **Structured Query Language (SQL)**. It covers the structural components of data storage, such as **Tables**, **Primary Keys**, and **Foreign Keys**, while providing a deep dive into **CRUD Operations** (Create, Read, Update, Delete).
Furthermore, the content explores advanced data manipulation using **Clauses**, **Operators**, and **Functions**, enabling professionals to efficiently retrieve, filter, and analyze vast datasets for both offensive vulnerability research and defensive security auditing.

This **documentation** and **learning approach** is strongly inspired by the structured, hands-on content provided by TryHackMe, which significantly contributed to practical understanding of "SQL Fundamentals" concepts.

DOCUMENTED BY

Kaustubh Ghosalkar aka "WEIRDO"

LINKEDIN GITHUB

1. Databases 101: Essential Concepts

A **Database** is an organized collection of structured information. In security, databases store everything from user credentials and logs to malware signatures.

Database Types

- **Relational Databases (SQL):** Store data in structured tables with fixed rows and columns. Ideal for consistent data like financial transactions.
- **Non-Relational Databases (NoSQL):** Store data in non-tabular formats (like JSON-style documents). Ideal for variable data like social media posts.

The Table Structure

- **Columns:** Define the category of data (e.g., username, password). Must have a defined **Data Type** (Integer, String, Date).
- **Rows:** A single entry or record within the table.
- **Primary Key:** A unique identifier for a record (e.g., user_id). Only one per table.
- **Foreign Key:** A column that links to a Primary Key in another table, creating a relationship between datasets.

2. Database and Table Statements

Interaction with a database happens through a **Database Management System (DBMS)** like MySQL or PostgreSQL.

Command Section: Database Management

Command Syntax	Description
CREATE DATABASE thm_db;	Creates a new database instance.
SHOW DATABASES;	Lists all available databases on the server.
USE thm_db;	Sets the specified database as the active environment.
DROP DATABASE thm_db;	Permanently deletes the database and all its contents.

Command Section: Table Management

- **CREATE TABLE inventory (id INT, item VARCHAR(255));** — Defines a new table structure.
- **DESCRIBE inventory;** — Displays the column names and data types of a table.
- **ALTER TABLE inventory ADD price INT;** — Modifies an existing table structure.
- **SHOW TABLES;** — Lists tables within the active database.

3. CRUD Operations

CRUD represents the four fundamental actions for data management.

Command Section: Data Manipulation

1. **Create (INSERT):** Adds a new row to the table.
   - *Example:* INSERT INTO books (id, title) VALUES (1, 'Cyber Security 101');
2. **Read (SELECT):** Retrieves data.
   - *Example:* SELECT * FROM books; (The * symbol retrieves **all** columns).
3. **Update (UPDATE):** Modifies an existing record.
   - *Example:* UPDATE books SET title = 'SQL Basics' WHERE id = 1;
4. **Delete (DELETE):** Removes a record.
   - *Example:* DELETE FROM books WHERE id = 1;

4. SQL Clauses: Refining Queries

Clauses specify criteria for how data is retrieved or sorted.

Clause	Purpose	Beginner Example
DISTINCT	Returns only unique values.	SELECT DISTINCT category FROM products;
ORDER BY	Sorts results.	SELECT * FROM users ORDER BY age ASC; (**ASC** = Low to High; **DESC** = High to Low).
GROUP BY	Aggregates data by column.	SELECT category, COUNT(*) FROM books GROUP BY category;
HAVING	Filters results **after** aggregation.	SELECT name FROM tools GROUP BY name HAVING COUNT(*) > 1;

5. Operators and Functions

Operators add logic to queries, while functions automate calculations on the data.

Logical Operators

- **LIKE**: Pattern matching using wildcards (e.g., % matches any characters).
- **AND / OR**: Combines multiple conditions.
- **BETWEEN**: Selects values within a specific range.
- **NOT**: Reverses the condition.

Comparison Operators

Used to compare values: **=** (Equal), **!=** (Not Equal), **<** (Less Than), **>** (Greater Than).

SQL Functions

- **CONCAT()**: Joins strings together (e.g., CONCAT(first_name, last_name)).
- **COUNT()**: Returns the number of rows.
- **SUM()**: Adds numeric values in a column.
- **MAX() / MIN()**: Finds the highest or lowest value in a column.

WEIRDO

Conclusion and Cheatsheet

Conclusion

SQL is a highly readable, English-like language that serves as the backbone of data interaction. Relational databases provide the reliability and speed needed for high-stakes environments like e-commerce and security auditing. By mastering **CRUD operations**, **Clauses**, and **Functions**, a user can effectively manage massive volumes of data, which is a foundational skill for understanding advanced topics like **SQL Injection** and database hardening.

Fast Revision Cheatsheet

- **SQL:** Structured Query Language for relational databases.
- **Primary Key:** Unique ID for every row.
- **SELECT:* Get everything from a table.
- **WHERE:** The standard way to filter results before they are returned.
- **LIKE '%word%':** Finds any string containing "word".
- **VARCHAR (255):** A text field limited to 255 characters.
- **INT:** An integer (number) field.
- **Port 3306:** The default port for MySQL.
- **Wildcards:** Use % for multiple characters and _ for a single character in LIKE queries